CSE234 Project 1
RAG-and-Context-Engineering

Lucheng Wang [*1] **Ziang Li** [*1]

1. Data Creation Methodology

Questions were selected to reflect realistic user needs when learning and applying RapidFire AI, especially for RAG and context-engineering workflows. Rather than focusing only on direct lookup questions, the set includes questions that require understanding how different parts of the framework connect, such as `RFLangChainRagSpec`, `run_evals`, config generators, user-provided eval functions, online aggregation, dashboards, Interactive Control Operations, vector stores, reranking, and troubleshooting.

The final 40-question golden Q&A set covers several question types. Some questions are factual or API-focused, asking about accepted input shapes, parameter roles, vector store modes, and metric aggregation behavior. Others are conceptual, asking why features such as online aggregation, sharded evaluation, or Interactive Control Operations matter. The set also includes procedural questions about setup, monitoring, troubleshooting, and recovering from interrupted experiments, as well as comparative and synthesis-oriented questions about how RapidFire AI components work together.

For each question, the reference answer was written using only information supported by the provided `.rst` documentation. The answers include the key facts, constraints, and relationships needed to fully answer the question, while avoiding unsupported outside assumptions.

Each Q&A pair follows the released validation set schema: `question_id`, `question`, `reference_answer`, and `source_evidence`. Each `source_evidence` entry records the exact `.rst` filename and inclusive line range supporting the answer. Each question has between 1 and 4 evidence spans, keeping the spans compact enough to avoid unrelated neighboring content while still fully supporting the reference answer.

Several quality checks were applied during construction. Each question was reviewed to ensure that it was answerable from the cited documentation, non-trivial, and not dependent on outside knowledge. Reference answers were checked against the selected evidence, and questions with ambiguous wording or overly broad evidence were revised or removed.

The dataset was improved through multiple iterations. Multi-source questions were added to test cross-document retrieval, overlap among questions about similar RapidFire AI concepts was reduced, wording was refined for clarity, and evidence spans were adjusted to improve precision. These iterations made the final golden set more balanced, less repetitive, and better suited for evaluating both retrieval quality and answer completeness.

In addition to the released judge metrics, two diagnostic LLM-as-judge dimensions were tested: Conciseness and Relevance. Conciseness checks whether an answer communicates the needed information efficiently without unnecessary repetition or padding. Relevance checks whether the answer directly addresses the question without drifting into tangential information. These metrics were used as diagnostic tools rather than replacements for Correctness, Faithfulness, and Completeness, because they target failure modes that can still appear in grounded answers.

2. Final Pipeline Architecture

The final system is implemented as an automated and reproducible command-line RAG pipeline rooted at `main.py`. Given an input JSON file of questions, the pipeline retrieves relevant chunks from the RapidFire AI `.rst` documentation corpus, generates grounded answers through an OpenAI-compatible TritonAI endpoint, and writes outputs in the required schema with `question_id`, `answer`, `retrieved_context`, and `sources`. The pipeline behavior is controlled through CLI arguments so that the selected configuration from experimentation can be rerun consistently by the autograder.

The pipeline begins by parsing CLI arguments into a `RAGConfig` object. This configuration object validates the corpus directory, vector store backend, retriever algorithm, embedding model, chunking settings, top-k value, reranking settings, context budget, PGVector connection set-

[*]Equal contribution [1]Department of Computer Science, University of California, San Diego, California, United States. Correspondence to: Lucheng Wang <luw033@ucsd.edu>, Ziang Li <zil165@ucsd.edu>.

tings, prompt strategy, and generation settings. This makes the execution flow reproducible because each run is tied to an explicit set of command-line parameters.

The corpus processing stage loads the RapidFire AI documentation files from `sourcedocs/` and converts them into LangChain-style `Document` objects. Two chunking strategies are supported. The line-based strategy splits each `.rst` file into fixed overlapping line windows. The section-based strategy detects reStructuredText section headings and keeps chunks aligned with documentation sections when possible; oversized sections are still split using the same line-window overlap rule. Each chunk stores metadata for the source filename, path, start line, and end line, which is later used to produce evaluator-compatible source citations.

The embedding stage converts documentation chunks and input questions into dense vectors using the configured SentenceTransformers model, with `sentence-transformers/all-MiniLM-L6-v2` as the default. For FAISS, chunk embeddings are generated and indexed during the pipeline run. For PGVector, corpus embeddings can be built ahead of time and stored in a persistent PostgreSQL collection, while query embeddings are generated at retrieval time with the same embedding model.

The retrieval stage supports dense, lexical, and hybrid retrieval. Dense retrieval uses FAISS or PGVector as the vector store backend. FAISS is built in memory during a run, while PGVector can reuse a prebuilt persistent collection in read mode. BM25 lexical retrieval is also supported for documentation questions involving exact API names, function names, configuration keys, and other precise terminology. The hybrid retriever combines vector similarity and BM25 results using reciprocal-rank fusion. Optional cross-encoder reranking first retrieves a larger candidate set and then selects the final top-k chunks.

After retrieval, the selected chunks are serialized into a prompt-ready context string. Each serialized document includes a document index, source file, line range, and chunk content. The context serializer estimates token length with `tiktoken` when available and uses a rough character-based estimate otherwise. By reserving part of the prompt budget for instructions and the question, the pipeline keeps the retrieved context within the configured budget while preserving source metadata for evaluation.

The generation stage uses a configurable prompt strategy controlled by `--prompt_strategy`, including `strict`, `detailed`, and `source_aware`, with `strict` as the default. These strategies differ in how much they encourage completeness or source awareness, but they share the same core constraint: the model should answer only from the retrieved context and avoid unsupported claims. Generation is performed through an OpenAI-compatible client using the configured TritonAI endpoint, API key path, generation model, and selected prompt strategy. The current implementation calls the model with temperature `0.0` to reduce run-to-run variation. For retrieval-only experiments, `--dry_run_generation` skips the LLM call and writes a fixed placeholder answer while still producing retrieved contexts and source spans.

The output stage writes one JSON object per question. Each object contains the `question_id`, generated answer, serialized `retrieved_context`, and a `sources` list with filename and line-span citations. Before saving, the output module validates that each entry contains a non-empty context and at least one well-formed source span. This keeps the submitted pipeline compatible with the project evaluation schema, while RapidFire AI experiment logs and configuration comparisons remain separate from the final automated inference script.

3. Experimentation Methodology

We used a sequential grid-search methodology instead of running a full Cartesian grid over all RAG knobs. A complete grid would have been prohibitively expensive because the search space included chunking strategy, chunk size, overlap, top-k, retriever algorithm, reranking, and prompt strategy. Therefore, we varied one family of knobs at a time while keeping the previously selected best settings fixed. After each stage, we selected the best-performing configuration according to retrieval metrics and used it as the baseline for the next stage.

For the early stages, we used RapidFire AI to run multi-config experiments over configurations that could be reliably represented and logged, namely chunking and top-k. In these stages, we programmatically constructed multiple `RFLangChainRagSpec` leaf configurations, similar to an `RFGridSearch`, and used RapidFire's multi-config experiment runner to evaluate them in parallel. To make the experiment traceable, we also saved an explicit project configuration table for each RapidFire stage. This table included `chunk_strategy`, `chunk_size`, `chunk_overlap`, `retriever_alg`, `top_k`, `search_k`, `prompt_strategy`, and `num_chunks`. This additional logging was necessary because RapidFire's dashboard serialization only exposed generic fields such as `search_cfg`, while our project-specific chunking knobs were hidden inside the custom document loader.

We did not use RapidFire AI for every later stage. During the retriever and reranker experiments, we observed that RapidFire's serialized configuration and internal caching path did not reliably preserve our custom project retriev-

Table 1. Stage 1 experiment results: chunking ablation. All configurations in this stage used `retriever_alg=similarity`, `top_k=5`, and reranking disabled. We compare line-based and section-based chunking with different chunk sizes and overlaps. The best configuration is line chunking with chunk size 10 and overlap 5.

Rank	Strategy	Size	Overlap	F1@5	P@5	R@5	Retrieval Score
1	`line`	10	5	0.5540	0.4815	0.7278	0.5878
2	`line`	20	5	0.4902	0.4000	0.7056	0.5319
3	`section`	10	5	0.4857	0.4000	0.6944	0.5267
4	`line`	30	5	0.4575	0.3407	0.7611	0.5198
5	`section`	20	5	0.4308	0.3259	0.7167	0.4911
6	`section`	30	5	0.4130	0.3037	0.7056	0.4741
7	`section`	50	10	0.3908	0.2815	0.6944	0.4556
8	`section`	100	20	0.3863	0.2815	0.6722	0.4467
9	`line`	100	20	0.3527	0.2593	0.6111	0.4077
10	`line`	50	10	0.3597	0.2741	0.5833	0.4057

ers. For example, different retriever settings could appear correctly in our preview table but still be displayed as the same native RapidFire `search_cfg` in the dashboard. To avoid reporting results from a potentially shared or incorrectly cached retriever, we ran the retriever, reranker, and prompt-strategy ablations directly through `main.py` using a shell script. This ensured that these later experiments used the same execution path as the final submitted pipeline.

The sequential ablation stages were:

1. **Chunking ablation:** compared two chunking strategies, `line` chunking and `section` chunking. For chunk size, we evaluated $\{10, 20, 30, 50, 100\}$. We used overlap 5 for chunk sizes 10, 20, and 30; overlap 10 for chunk size 50; and overlap 20 for chunk size 100.
2. **Top-k ablation:** varied `top_k` over $\{3, 5, 7\}$.
3. **Retriever ablation:** compared `similarity`, `mmr`, `bm25`, and `hybrid`.
4. **Reranking ablation:** compared reranking disabled against cross-encoder reranking enabled.
5. **Prompt ablation:** compared `strict`, `detailed`, and `source_aware` prompting with the LLM judge enabled.

4. Results and Analysis

4.1. Results and Analysis

Tables 1–5 summarize the staged ablation results. Overall, the final pipeline we selected used line-based chunking with chunk size 10 and overlap 5, hybrid retrieval, `top_k=3`, reranking disabled, and the `strict` prompt strategy. This configuration was selected because it gave the strongest overall balance between retrieval quality and generation quality in our staged search.

Table 2. Stage 2 experiment results: top-k ablation. All configurations in this stage used the best chunking setup from Stage 1, `chunk_strategy=line`, `chunk_size=10`, and `chunk_overlap=5`, with `retriever_alg=similarity` and reranking disabled. We varied `top_k` over $\{3, 5, 7\}$. The best configuration used `top_k=3`.

Rank	Top-k	Search-k	F1@5	P@5	R@5	Retrieval Score
1	3	3	0.5252	0.4444	0.7167	0.5621
2	5	5	0.4416	0.3378	0.7500	0.5098
3	7	7	0.4416	0.3378	0.7500	0.5098

Table 3. Stage 3 experiment results: retriever ablation. All configurations in this stage used the best settings from previous stages: `chunk_strategy=line`, `chunk_size=10`, `chunk_overlap=5`, and `top_k=3`. Reranking was disabled for all runs. We compared `similarity`, `mmr`, `bm25`, and `hybrid` retrieval. The best configuration used the `hybrid` retriever.

Rank	Retriever	F1@5	P@5	R@5	Retrieval Score
1	`hybrid`	0.6207	0.5407	0.8000	0.6538
2	`similarity`	0.5540	0.4815	0.7278	0.5878
3	`bm25`	0.4763	0.4222	0.6000	0.4995
4	`mmr`	0.4235	0.3111	0.7389	0.4912

Chunking ablation. The chunking configuration had a large effect on retrieval quality. In Stage 1, the best configuration was line-based chunking with chunk size 10 and overlap 5, which achieved F1@5 = 0.5540, P@5 = 0.4815, R@5 = 0.7278, and a retrieval score of 0.5878. In general, smaller chunks performed better than larger chunks. For example, line chunking with size 10 outperformed line chunking with size 50 and size 100 by a large margin. This suggests that, for our documentation-style corpus, smaller chunks were more likely to isolate the exact relevant lines needed to answer a question. Larger chunks often contained more surrounding context, but this extra context appeared to reduce precision by adding irrelevant text to the retrieved context.

Line-based chunking also generally performed better than section-based chunking in the top-ranked settings. The best section-based setting, section chunking with size 10 and overlap 5, reached a retrieval score of 0.5267, which was lower than the best line-based score of 0.5878. We expected section-aware chunking to help preserve semantic structure, but in this dataset many questions are tied to specific API details, parameter descriptions, or short documentation passages. For this type of evidence, smaller line-based chunks were more effective because they gave the retriever finer-grained units to match against the query.

Top-k ablation. In Stage 2, we fixed the best chunking configuration from Stage 1 and varied `top_k` over $\{3, 5, 7\}$. The best result used `top_k=3`, with F1@5 = 0.5252, P@5 = 0.4444, R@5 = 0.7167, and retrieval score = 0.5621. Increasing `top_k` to 5 or 7 increased recall slightly, but

Table 4. Stage 4 experiment results: reranking ablation. All configurations in this stage used `chunk_strategy=line`, `chunk_size=10`, `chunk_overlap=5`, `retriever_alg=similarity`, and `top_k=3`. We compared retrieval with reranking disabled against cross-encoder reranking using `BAAI/bge-reranker-base`. The reranked configuration achieved the best retrieval score.

Rank	Rerank	Reranker	F1@5	P@5	R@5	Retrieval Score
1	True	`BAAI/bge-reranker-base`	0.6362	0.5556	0.8389	0.6769
2	False	–	0.5540	0.4815	0.7278	0.5878

Table 5. Stage 5 experiment results: prompt ablation. All configurations used `chunk_strategy=line`, `chunk_size=10`, `chunk_overlap=5`, `retriever_alg=hybrid`, `top_k=3`, and reranking disabled. Since the retrieval configuration was fixed, all prompts shared the same retrieval metrics: F1@5 = 0.6207, P@5 = 0.5407, R@5 = 0.8000, and retrieval score = 0.6538. The final ranking is based on generation quality with the LLM judge enabled.

Rank	Prompt	Correct	Faith	Complete	Concise	Relevant	Gen. Score
1	`strict`	0.6889	1.0000	2.47/5	4.49/5	3.82/5	0.7689
2	`source_aware`	0.6444	1.0000	2.58/5	4.29/5	4.04/5	0.7653
3	`detailed`	0.6000	0.8889	2.67/5	4.02/5	3.89/5	0.7209

precision dropped substantially, leading to a lower overall retrieval score. This indicates that retrieving more chunks was not always beneficial. Since the context window is limited, adding more chunks can introduce distracting or redundant information, which hurts precision and can also make generation harder. Therefore, `top_k=3` gave the best precision–recall tradeoff in our setting.

Retriever ablation. In Stage 3, we compared `similarity`, `mmr`, `bm25`, and `hybrid` retrieval using the best chunking and top-k settings from the earlier stages. The hybrid retriever performed best, achieving F1@5 = 0.6207, P@5 = 0.5407, R@5 = 0.8000, and retrieval score = 0.6538. This was a clear improvement over pure similarity retrieval, which achieved a retrieval score of 0.5878. BM25 also performed reasonably well, but was weaker than similarity and hybrid overall. MMR performed the worst among the four retrievers, with a retrieval score of 0.4912.

The hybrid retriever likely worked well because the corpus contains both semantic concepts and exact lexical/API terms. Dense similarity retrieval is useful for matching paraphrased questions to semantically related documentation, while BM25 is useful for exact matches to function names, parameter names, and documentation keywords. Combining these two signals made retrieval more robust. In contrast, MMR may have over-emphasized diversity. For our task, retrieving several highly relevant chunks from the same documentation region is often more useful than retrieving diverse but less directly relevant chunks.

Reranking ablation. In Stage 4, we tested cross-encoder reranking using `BAAI/bge-reranker-base`. Reranking improved the similarity retriever substantially, increasing the retrieval score from 0.5878 to 0.6769. Precision, recall, and F1 all improved: F1@5 increased from 0.5540 to 0.6362, P@5 increased from 0.4815 to 0.5556, and R@5 increased from 0.7278 to 0.8389. This result shows that reranking can be highly effective because the first-stage retriever may retrieve useful candidates but not rank them optimally. The cross-encoder reranker can compare the query and candidate chunks more directly and reorder the retrieved set by relevance.

However, our final prompt-generation ablation used the hybrid retriever without reranking. We made this choice for two reasons. First, the reranking experiment was run on the similarity retriever, while the retriever ablation selected hybrid as the best base retriever. Therefore, the reranking result should be interpreted as strong evidence that reranking is promising, but not a complete test of the final hybrid pipeline. Second, reranking was much more expensive in runtime because it required an additional cross-encoder scoring step over the retrieved candidates for each query. To keep the final prompt ablation within our available time budget, we disabled reranking and used the best non-reranked retriever, `hybrid`. With more time or compute budget, the most important follow-up experiment would be to test `hybrid + reranker`, since this may combine the strongest base retriever with the strongest ranking method.

Prompt ablation and generation metrics. In Stage 5, we fixed the retrieval configuration to hybrid retrieval with line chunking, chunk size 10, overlap 5, `top_k=3`, and reranking disabled. Since retrieval was fixed, all prompt strategies had the same retrieval metrics: F1@5 = 0.6207, P@5 = 0.5407, R@5 = 0.8000, and retrieval score = 0.6538. Therefore, this stage measures the effect of prompting on generation quality rather than retrieval quality.

The `strict` prompt performed best, with generation score = 0.7689. It also achieved the highest correctness score, 0.6889, and perfect faithfulness, 1.0000. The `source_aware` prompt was very close, with generation score = 0.7653, and it achieved the highest relevance score, 4.04/5. The `detailed` prompt performed worst, with generation score = 0.7209. Although it had the highest completeness score, 2.67/5, it had lower correctness and faithfulness. This suggests that asking the model for more detailed answers can encourage longer responses, but those responses may include unsupported or less precise information. For this project, the stricter prompt was better because the task rewards grounded answers based only on retrieved context.

Impact of different knobs. The knobs with the largest impact on retrieval metrics were chunking, retriever algorithm, and reranking. Chunking controlled the granularity of retrievable evidence; small line chunks substantially improved precision and retrieval score. The retriever algorithm also had a major effect: hybrid retrieval improved the retrieval score from 0.5878 for similarity to 0.6538. Reranking had an even larger isolated effect on similarity retrieval, improving the retrieval score to 0.6769. Top-k had a more moderate but still important effect: `top_k=3` worked better than larger values because it reduced noisy retrieved context.

For generation metrics, the biggest tested knob was prompt strategy. Since retrieval was fixed in Stage 5, differences in generation score came from how the model used the same retrieved context. The strict prompt produced the best balance of correctness, faithfulness, conciseness, and relevance. The detailed prompt improved completeness slightly, but this came at the cost of correctness and faithfulness, which made it less suitable for this task.

Surprising observations. One surprising result was that smaller chunks performed better than larger or section-based chunks. We initially expected section-based chunking to preserve more semantic structure, but the results suggest that fine-grained line chunks were more effective for this documentation QA task. Another surprising result was that increasing `top_k` did not improve the retrieval score. Although larger `top_k` values slightly improved recall, they reduced precision enough to hurt the overall score. Finally, MMR performed worse than expected. While MMR is designed to improve diversity, this task appears to require concentrated evidence from the most relevant documentation regions rather than diverse evidence from many different regions.

Final pipeline. Based on these results, our final selected pipeline used line chunking with chunk size 10 and overlap 5, hybrid retrieval, `top_k=3`, reranking disabled, and the `strict` prompt. This pipeline was selected because line chunking gave the best chunking result, `top_k=3` gave the best precision–recall tradeoff, hybrid retrieval achieved the best base retriever score, and the strict prompt achieved the best generation score under the fixed retrieval setting.

Future work. If we had more time or budget, the first experiment we would run is `hybrid + reranker`. The reranker substantially improved similarity retrieval, while hybrid was the best base retriever, so combining them may produce the strongest retrieval pipeline. We would also tune the hybrid retriever weights instead of using a fixed combination of dense and BM25 retrieval. In addition, we would test more embedding models, vary the reranker fetch size, and evaluate whether a larger context budget changes the optimal `top_k`. Finally, we would run a full Cartesian grid over the most promising settings, including generation evaluation with the LLM judge, to verify whether the sequential search missed any interaction effects between chunking, retrieval, reranking, and prompting.

5. AI Tools Statement

AI assistants were used to help implement parts of the code, debug errors, and clean up code structure. The overall system design, experimental direction, and final configuration choices were made by the human team members. All AI-written code was reviewed, tested, and revised by the team before being included in the final project.